

Proje 3: Akıllı Veri Analitiği ve Makine Öğrenmesi Uygulaması

Şarkı Öneri Sistemi Proje Raporu

Alan	Açıklama
Proje konusu	Akıllı veri analitiği ve makine öğrenmesi tabanlı öneri sistemi
Uygulama adı	Song Recommender
Temel çıktı	Kullanıcının seçtiği şarkıya benzer 5 şarkının önerilmesi
Teknoloji odağı	Python, React/TypeScript, Flask, Spotify API, bulut uyumlu servis mimarisi

Yönetici Özeti

Bu rapor, verilen proje kapsamına uygun olarak geliştirilen şarkı öneri sisteminin veri analitiği, makine öğrenmesi, servis mimarisi ve kullanıcı arayüzü

1. Projenin Amacı ve Kapsamı

Bu projenin amacı, veri analitiği ve makine öğrenmesi yaklaşımını kullanarak kullanıcının sevdiği bir şarkıya benzer şarkıları otomatik biçimde önerebilen akıllı bir uygulama geliştirmektir. Proje, Python tabanlı modelleme, web servis katmanı, modern istemci arayüzü ve bulut platformlarına taşınabilecek bir dağıtım yaklaşımını birlikte ele alır.

Sistem, kullanıcıdan alınan şarkı adını veri setindeki kayıtlarla eşleştirir; önceden hesaplanan benzerlik matrisi üzerinden en yüksek skorlu şarkıları seçer ve her öneri için Spotify servisinden albüm görseli, sanatçı adı ve bağlantı bilgisi almaya çalışır.

- Veriden anlamlı örüntüler çıkararak öneri üretmek.
- Model çıktısını web servis aracılığıyla erişilebilir hale getirmek.
- Kullanıcı dostu bir arayüzle önerileri görsel olarak sunmak.
- Geliştirme ortamından bulut dağıtımına taşınabilecek bir mimari önermek.

2. Kullanılan Teknolojiler

Proje, verilen kapsam doğrultusunda veri işleme, makine öğrenmesi, backend servisleri, frontend arayüzü ve bulut dağıtım katmanlarını bir arada ele alır.

Katman	Kullanılan teknoloji	Projedeki rolü
Backend	Python, Flask, Flask-CORS	Öneri API'sini çalıştırır ve istemci isteklerine JSON yanıt üretir.
Veri / ML	Pickle model dosyaları, NumPy, veri işleme ve benzerlik matrisi üzerinden öneri hesaplaması yapar.	
Harici servis	Spotify API, Spotipy	Önerilen parçaların sanatçı, kapak görseli ve bağlantı bilgilerini sağlar.
Frontend	React, TypeScript, Axios	Kullanıcıdan arama alır, API'ye istek atar ve önerileri listeler.
Bulut hedefi	AWS, Azure veya Google Cloud	API, frontend ve veri dosyaları için ölçeklenebilir dağıtım ortamı sunar.

3. Sistem Mimarisi

Uygulama üç ana bileşenden oluşur: kullanıcı arayüzü, öneri API'si ve veri/model katmanı. Bu ayırım, hem geliştirmeyi sadeleştirir hem de sistemin bulut ortamına taşınmasını kolaylaştırır.

Adım	Bileşen	İşlem
1	React arayüzü	Kullanıcı şarkı adını girer ve arama isteğini başlatır.
2	Flask API	/recommend endpoint'i song parametresini alır ve doğrular.
3	Model katmanı	Şarkı veri setindeki indeks bulunur; benzerlik matrisi sıralanır.
4	Spotify entegrasyonu	Önerilen şarkılar için kapak, sanatçı ve Spotify bağlantısı alınır.
5	İstemci sunumu	En iyi 5 öneri eşleşme yüzdesiyle kullanıcıya gösterilir.

4. Veri Analitiği ve Model Yaklaşımı

Öneri sistemi, içerik tabanlı öneri mantığıyla çalışacak şekilde kurgulanmıştır. Veri setindeki her şarkı için tür ve benzeri özelliklerden elde edilen temsil üzerinden şarkılar arası benzerlik değerleri hesaplanır. Bu hesaplamanın sonucu similarity_matrix.pkl dosyasında saklanır; songs_df.pkl dosyası ise şarkı adları ve tür bilgileri gibi kayıtları içerir.

API çalıştığında bu iki dosya belleğe yüklenir. Kullanıcı bir şarkı gönderdiğinde sistem ilgili satır indeksini bulur, benzerlik skorlarını büyükten küçüğe sıralar ve ilk şarkının kendisi olma ihtimalini dışarıda bırakarak sonraki 5 kaydı öneri listesine ekler.

- Girdi: Kullanıcının yazdığı şarkı adı.
- İşleme: Veri setinde eşleşme kontrolü ve benzerlik skorlarının sıralanması.
- Çıktı: Şarkı adı, tür, benzerlik skoru ve Spotify bilgileri içeren JSON öneri listesi.
- Hata durumu: Şarkı veri setinde yoksa kullanıcıya anlaşılır hata mesajı döndürülür.

5. Backend Uygulaması

Backend katmanı Flask ile geliştirilmiştir. CORS desteği etkinleştirildiği için frontend farklı porttan API'ye istek gönderebilir. Spotify API kimlik bilgileri .env dosyasından okunur; böylece hassas bilgiler kaynak kodun dışında tutulur.

Backend mantığı kısa ve anlaşılır tutulmuştur. Buna karşın üretim ortamı için giriş doğrulama, loglama, oran sınırlama, model dosyası sürümleme ve Spotify API hatalarına karşı daha ayrıntılı istisna yönetimi eklenmelidir.

Endpoint	Metot	Görev
/recommend?song=...	GET	Girilen şarkıya göre en benzer 5 şarkıyı JSON olarak döndürür.
/songs?query=...	GET	Şarkı adı içinde arama yaparak en fazla 10 eşleşme döndürür.

6. Frontend Uygulaması

Frontend, React ve TypeScript ile geliştirilmiştir. Kullanıcı arama kutusuna bir şarkı adı girer; Find düğmesi veya Enter tuşu ile API çağrısı yapılır. Axios, Flask API ile haberleşmek için kullanılır. Sonuçlar kart yapısında gösterilir ve Spotify verisi varsa albüm kapağı, sanatçı adı ve bağlantı da ekrana basılır.

- Boş arama gönderimi engellenir.
- Yükleme durumunda kullanıcıya görsel geri bildirim verilir.
- Hata durumunda API yanıtı arayüzde gösterilir.
- Benzerlik skoru yüzde formatına çevrilerek kolay anlaşılır hale getirilir.

7. Bulut Platformu ve Dağıtım Planı

Proje mevcut haliyle yerel geliştirme ortamında çalışır; ancak mimari buluta taşınmaya uygundur. Küçük ölçekli akademik teslim için en pratik seçenek, frontend'in statik olarak yayınlanması ve Flask API'nin konteyner olarak Cloud Run, Azure Container Apps veya AWS Elastic Beanstalk üzerinde çalıştırılmasıdır.

- API dağıtımı: AWS Elastic Beanstalk/ECS, Azure App Service/Container Apps veya Google Cloud Run.
- Model/veri dosyaları: S3, Azure Blob Storage veya Google Cloud Storage.
- Analitik depo: PostgreSQL, BigQuery, Cloud SQL veya benzeri yönetilen veri hizmetleri.
- ML geliştirme: SageMaker, Azure Machine Learning veya Vertex AI.

8. Güvenlik, Performans ve Sürdürülebilirlik

Uygulamanın güvenli ve sürdürülebilir biçimde çalışması için API anahtarlarının gizli tutulması, veri dosyalarının sürümlenmesi, kullanıcı girdilerinin doğrulanması ve hata mesajlarının kontrollü biçimde döndürülmesi gerekir.

Konu	Mevcut durum	Önerilen iyileştirme
Gizli bilgiler	.env üzerinden okunuyor	Bulutta Secret Manager, Key Vault veya AWS Secrets Manager kullanılmalı.
Model dosyaları	Yerel pickle dosyaları	Sürüm numarasıyla nesne depolamada tutulmalı.
Performans	Benzerlik matrisi bellekte	Büyük veri setlerinde yaklaşık en yakın komşu indeksleme veya önbellek kullanılması.
API dayanıklılığı	Temel hata yanıtları var	Spotify zaman aşımı, kota ve ağ hataları için fallback eklenmeli.

9. Test ve Doğrulama

Sistemin doğrulanması hem fonksiyonel hem de kullanıcı deneyimi açısından yapılmalıdır. Temel test senaryoları aşağıdaki gibidir:

- Veri setinde bulunan bir şarkı adı girildiğinde 5 öneri dönmesi.
- Veri setinde bulunmayan şarkı adı girildiğinde 404 hata mesajı dönmesi.
- Boş song parametresi gönderildiğinde 400 hata mesajı dönmesi.
- Spotify API yanıtı yoksa önerinin temel şarkı ve tür bilgileriyle yine gösterilebilmesi.
- Frontend tarafında loading, error ve successful result durumlarının ayrı ayrı denenmesi.

10. Elde Edilecek Çıktılar

- Makine öğrenmesi tabanlı şarkı öneri modeli ve benzerlik skorları.
- Bulut üzerinde çalıştırılacak Flask tabanlı öneri servisi.
- React/TypeScript ile geliştirilmiş kullanıcı arayüzü.
- Spotify entegrasyonu ile zenginleştirilmiş öneri kartları.
- Veriden bilgi çıkarma, sınıflandırma/benzerlik analizi ve tahmin mantığını gösteren akademik proje çıktısı.

11. Sonuç

Song Recommender uygulaması, Proje 3 kapsamında istenen akıllı veri analitiği ve makine öğrenmesi uygulaması hedefine uygun bir örnektir. Sistem; veri seti, benzerlik matrisi, API servisi ve web arayüzünü bir araya getirerek uçtan uca çalışan bir öneri mekanizması sunar. Mevcut sürüm akademik teslim için yeterli bir prototip niteliğindedir; bulut dağıtımı, otomatik tamamlama, kapsamlı testler ve model güncelleme süreci eklendiğinde daha güçlü bir üretim uygulamasına dönüşebilir.

Ek: Kod Tabanı Kanıtları

Dosya	Raporda kullanılan kanıt
song-recommender-backend/app.py	Flask API, Spotify entegrasyonu, pickle model dosyaları, /recommend ve /songs endpoint'leri.
my-app/src/App.tsx	React/TypeScript arayüzü, Axios API çağrısı, öneri kartları ve hata/yüklenme durumları.
song-recommender-backend/similarity_matrix.pkl	Öneri skorlarının üretildiği kayıtlı benzerlik matrisi.
song-recommender-backend/songs_df.pkl	Şarkı adları ve tür bilgilerini içeren veri dosyası.